

Tutor conversations: plan review

Captured (definition-capture) · 7 October 2026 · Branch `tutor-conversations` · Plan `Docs/plans/2026-10-07-tutor-conversations.md` · Spec `Docs/tutor-conversations.md`

Two independent read-only reviews of the implementation plan, asked to check efficiency, security, stability and potential data failure points when many people use the app. Both reviewers worked under the ponytail discipline. Their reports are reproduced below as they were delivered; neither edited any file.

1. ai-architect review

I checked the plan against the real files. On a busy day the riskiest thing in it is the time budget: the new steps no longer fit in 60 seconds.

High

- 1. Task 4 Step 3 and Task 5 Step 3: a question can now take longer than Vercel's 60-second limit.**
Today the model call has 55s and little else runs. The plan adds up to 15s to embed the question before the model and another 15s to embed the answer after it, so a question can need 85s and a merge 70s. When Vercel stops the function at 60s, no catch block runs. The learner loses an answer they paid for, and a first question can leave an empty conversation behind. Fix: give each route one deadline (about 50s) and size every timeout from what's left. Embedding should get only a few seconds and be skipped if time is short. Cheapest option: save the question's embedding (already made for memory) instead of making a second one after the model. Meaning search would then cover questions only, and keyword search still covers answers.
- 2. Task 1: storage on the free plan.** Each exchange stores a 4KB vector, the reply JSON, a copy of the answer text, and a stored search column with its index. That's roughly 12 to 15KB per exchange, so the 500MB free database fills at about 35,000 exchanges across all accounts, a few hundred active learners. Fix: use pgvector's half-precision vector type. Replace the stored search column with an index on the same expression. Add this limit to the spec's Weak points.

Medium

- 3. Task 4 Step 3: avoidable steps in a row.** The current plan makes about 12 round trips one after another. Reading the conversation, the account's state, the recent exchanges, the rule titles, and the question's embedding plus memory search can all start together before the reservation. They are reads, and search is already free to use anyway. Making the conversation can run alongside the answer's embedding.
- 4. Task 2 and Task 4: each question costs about three times as much.** The history limit stays at 10 but now counts exchanges, not messages, so it doubles, and memory adds 5 more exchanges on a costly model. Each saved answer can be up to 32,000 characters. Fix: keep history at 5 exchanges and put a character cap on history plus memory.
- 5. Task 9 Step 5: a full page refresh after every answer.** It re-reads the whole conversation and the sidebar and sends them back to the browser each time. It's also a data bug: if another tab deleted the conversation, the refresh swaps the chat panel for an empty one and the answer just paid for disappears. Fix: refresh only when the answer was saved. Separately, the "Answer in" switch goes back to its default after a new conversation gets its address.

6. **Task 3 and Task 7: tick boxes on answers that can't be merged.** After the key is rotated, old answers still show tick boxes, but merging them fails with "The tutor could not answer". Fix: the page should only show a tick box on answers whose signature checks out.
7. **Task 1 and Task 6: search does more work than it needs.** The search function returns the full reply JSON (up to 200KB a row) even though memory and the sidebar never use it; drop it from what search returns. The keyword index covers every account at once, so check its query plan at realistic volume. Any search box (keyword or meaning) can hammer the one shared OpenRouter key and get it rate-limited, which then breaks paid questions too.

Low

- Task 3: if the exchange's save fails after the conversation is made, an empty named conversation stays in the sidebar. Embedding before making the conversation shrinks that window.
- Task 9 Step 7: the page loads the conversation in two steps; all five reads could run at once. A whole conversation is loaded with no upper limit.
- Task 9 Step 6: pressing Enter in Rename may also fire the blur handler, which saves the same name twice. Harmless.

Fine as is

- No other account can be reached through a conversation id or exchange ids. The database's ownership checks and two-part links stop it, and the rehearsal tests it.
- Memory only includes the account's own answers, and only signed ones.
- Two tabs asking in one conversation can't corrupt anything.
- A double-click can't send two merges, because the button is disabled straight away.
- Renaming a conversation that was just deleted is a harmless no-op.
- The links between rules and conversations wait for the rule's own save. If that save failed, the database refuses the link.
- The deletes clean up after themselves through the database's cascades, and every one has an index behind it.

Files: `Docs/plans/2026-10-07-tutor-conversations.md`, `src/lib/tutorServer.ts`,
`src/app/api/tutor/route.ts`

2. Supabase review (supabase, supabase-postgres-best-practices and supabase-security skills)

I reviewed the plan without editing anything. The schema is sound and RLS is tight, but there are two things to fix before anyone starts building: the rehearsal can't run as written, and search has no relevance cut-off.

High

1. **Task 1 Step 3: the rehearsal fails at setup.** It inserts two grammar items without `blocks`. The `items_grammar_fields` check (grammar_rules migration) requires `blocks` to be set for grammar items, so the script aborts before the first test runs. Fix:

```
insert into public.items (id, user_id, item_type, title, blocks) values (... , 'grammar', 'Dative', '[]'), (... , 'grammar', 'Genitive', '[]');
```

2. **Task 1 search_tutor and Review Focus 4: meaning search always returns rows.** The semantic CTE has no distance cut-off. Whenever an embedding is present (the normal case), even "words that occur nowhere" return the nearest exchanges. The rehearsal's `'zzzz'` case proves it. The same goes for memory in Task 4, which injects 5 unrelated earlier exchanges into every question, costing tokens and adding noise. Fix: add to the semantic CTE's `where`:

```
and (e.embedding operator(extensions.<=>) query_embedding) < 0.5
```

- Mark it `-- ponytail:` and tune the 0.5 in Task 11 Step 5 against real distances.
- Add a rehearsal case: a far vector such as `array_fill(-0.1, array[1024])` with `'nowhere'` returns nothing.

Medium

3. **Task 1: the B-account cross-account test tests nothing.** B inserts into a zero uuid, not A's conversation. Fix: in A's block, `perform set_config('test.conv', conv::text, true);`. Then in B's block:
 - insert an exchange with `current_setting('test.conv')::uuid`, expecting `foreign_key_violation`;
 - insert `tutor_conversation_rules (current_setting('test.conv')::uuid, b1)`, expecting `foreign_key_violation`;
 - `update tutor_conversations set name='x'` and check that `get_diagnostics ... row_count = 0`.
4. **Storage against the free plan's 500 MB.** The stated scale doesn't fit the free plan.
 - Per exchange: about 4.1 KB of embedding, plus reply jsonb, plus `answer_text` (which duplicates the reply's text), plus a stored tsvector, GIN entries and the btree. That's roughly 10 to 15 KB typical, and up to 240 KB worst case.
 - That allows about 40k exchanges across all accounts, versus millions for "thousands of users × thousands of exchanges".
 - A paid account at 30 a day adds about 400 KB a day.
 - Past 500 MB the project goes read-only, so every write in the app fails, not just the tutor's.
 - Any account can also insert rows directly with the publishable key, with no usage check, at 200 KB each. One account could fill the project. Items have the same exposure today, but this is bigger.

- Smallest fix, done now because changing it later is a table rewrite: `embedding_extensions.halfvec(1024)`. The parameter type and grant signatures become `extensions.halfvec`; `<=>` and the JSON-string input work unchanged. That halves vector storage and scan I/O at negligible retrieval cost.
- Also lower the reply cap to `octet_length(reply::text) <= 64000`. `answer_text` is at most 32000 characters, so 200 KB is slack.
- Add a Weak points line: the free plan holds about 40k exchanges in total, so watch Dashboard → Usage and move to Pro before about 350 MB. Direct inserts are bounded only by these size caps.

5. Performance of the exact scan.

- Each search or question computes distance over all of the account's embedded rows. A `vector(1024)` is 4.1 KB, above the 2 KB TOAST threshold, so every row is a TOAST fetch: about 20 MB read per query at 5,000 exchanges.
- On the free plan's small cache and burst-limited disk, cold reads can take seconds, and every question pays it twice (memory, then search).
- At the database sizes the free plan can actually hold, the table is small enough that this is fine.
- `halfvec(1024)` is still 2,056 bytes, just over the threshold. Add `alter table public.tutor_exchanges alter column embedding set storage main;` to keep it inline.
- Add a Task 11 check: seed 5k rows locally and run `explain (analyze, buffers)` on `search_tutor`.

6. Task 3 / spec: changing the embedding model. Swapping to another model with the same dimension silently mixes two vector spaces.

- Rows whose embedding failed (null) can never be backfilled: there is no update grant and no secret key.
- Fix: a comment on `EMBEDDING_MODEL` saying that changing it needs a migration running `update public.tutor_exchanges set embedding = null`, so old rows fall back to keyword search.
- Accept that there's no backfill and say so in Weak points.

Low

- score**: the expression is numeric while the declared type is double precision. PG 13+ casts it automatically, but write `(coalesce(...) + coalesce(...))::double precision` to make it explicit.
- Account deletion**: the `tutor_conversation_rules.user_id → auth.users` cascade has no index leading with `user_id`, so it seq-scans when an account is deleted. Either drop that direct reference (the composite foreign key to conversations already cascades) or add `create index on public.tutor_conversation_rules (user_id)`.
- Search payload**: `search_tutor` returns `reply` (up to 200 KB) for 30 rows on every sidebar keystroke, though the route only uses `id`, `name`, `question` and `answer_text`. Acceptable for now, since `toStored` needs `reply`; the 64 KB cap above bounds it.
- Task 11 Step 4**: before `db push`, check Dashboard → Extensions that `vector` isn't already enabled in `public`. If it is, `if not exists` skips and `extensions.vector` won't resolve. Also run `npx supabase migration list` first to confirm only this migration is pending.
- Task 11 Step 7**: after the drop, an Instant Rollback in Vercel to a pre-merge deployment would hit a missing table. Note it in HANDOFF.
- Task 10 seed**:
 - `removeSeeded` ignores its delete errors. Throw on them so leftovers don't break later runs.
 - Each search keystroke in e2e calls OpenRouter's embeddings endpoint with the real key, so "costs nothing" is a fraction of a cent, not zero.

What is fine

- **Migration syntax:** `create extension ... with schema extensions` and `operator(extensions.<=>)`; the generated `tsvector` with a `'simple'::regconfig` cast; `jsonb_typeof` and `octet_length(reply::text)` in a check; `default auth.uid()`; the mixed column-level `grant` syntax, and the `revoke/grant` and `has_function_privilege` signatures with `extensions.vector`.
- **Search function internals:** under `search_path = ''`, `ts_rank_cd`, `websearch_to_tsquery` and `@@` resolve from `pg_catalog`; `row_number()` over with `order by rank limit` inside the CTEs works; the full outer join on bigint equality is hash/merge-joinable; in a SQL function only input parameters are visible by name, so the output column names cause no ambiguity; punctuation-only queries give an empty `tsquery` (a NOTICE, no error); `array_fill(0.1, array[1024])::extensions.vector` works (pgvector casts `numeric[]`).
- **Rehearsal error codes:** RLS rejection gives `insufficient_privilege` (42501). `delete from public.items` works as authenticated, since `items` has a delete grant and policy. The anon checks hit 42501 because anon gets no table grants and no execute on `search_tutor`.
- **Cross-account isolation:** composite foreign keys including `user_id` make cross-account links impossible and leak no existence oracle. Cascades run as the owner. `search_tutor` is security invoker under RLS.
- **Column grants:** an account setting a future `updated_at` only reorders its own sidebar. Every policy uses `(select auth.uid())`.
- **Indexes:** they cover the sidebar, latest-N (backward scan on `(user_id, conversation_id, id)`), by-ids (primary key), linked rules (primary key prefix), and the cascades from `items` and `conversations`.
- **PostgREST:** a JSON-stringified vector works on both insert and rpc; `.in("id", ids)` with numbers and `.order().limit()` behave as expected; `maybeSingle` returns null on zero rows; 23503 and 23505 come back in `error.code`; an insert with `return=minimal` needs no select policy.
- **Release order:** additive. Old production code keeps working against the new schema. The only loss is `conversation_messages` rows written between the push and the deploy, which is the owner's decision.

3. Decisions taken after the reviews

Recorded on 7 October 2026 and applied to the plan and spec:

- **All high, medium and low items are applied:** a 55-second time budget with 5-second embeddings and parallel reads; half-precision vectors kept in the row; a keyword index on an expression instead of a stored column; a 64 KB reply cap; a closeness cut-off on meaning search (0.5, tuned during the real call); history of 5 exchanges with a 12,000-character cap on memory; refresh only after a saved answer, and the Answer in choice kept across a new conversation's address; search returning no reply JSON; the rehearsal fixes; the embedding-model note; and the small items.
- **Signatures get their own secret,** `TUTOR_SIGNING_SECRET`, so replacing the OpenRouter key never leaves older answers unmergeable. Only answers whose signature verifies get a tick box.
- **Per-account cap:** 2,000 saved exchanges and 500 conversations, enforced by the database on every insert (so direct inserts with the publishable key are held to it too) and checked by the routes before a message is spent.
- **Search limit:** 100 searches per account per hour, counted in an insert-only log that a nightly `pg_cron` job clears of rows older than a day.
- **The merged rule follows the Answer in switch,** as a question does.